

Git & GitHub: A Complete Guide for Game Developers

From "what even is a commit" to rewriting history, recovering lost work, and surviving a merge conflict in a Unity scene file.

How to use this guide: Part 1–5 is the foundation. Part 6–7 is GitHub and team workflow. Part 8 is game-dev-specific (binaries, LFS, engine setup). Part 9 is a scenario library — the "something went wrong, what do I type" section. Parts 10–13 are reference: error decoder, cheat sheet, drills, glossary.

If you're impatient: read Part 2, Part 4 (branches), Part 5 (conflicts), then jump to Part 9 and use the rest as reference.

Table of Contents

1. [The mental model](#)
 2. [Setup and first repo](#)
 3. [The three trees and the core loop](#)
 4. [Branches: the real explanation](#)
 5. [Remotes, fetch, pull, push, merge, conflicts](#)
 6. [Rebase, history rewriting, and undo](#)
 7. [GitHub: PRs, forks, reviews, automation](#)
 8. [Game dev: binaries, LFS, engines](#)
 9. [Scenario library](#)
 10. [Error message decoder](#)
 11. [Cheat sheet](#)
 12. [Practice drills](#)
 13. [Glossary](#)
-

Part 1 — The mental model

Git is not a backup tool. It's a database of snapshots.

Most people picture git as "saving versions of files." That model breaks the moment you hit a conflict. Here's the accurate one:

Every time you commit, git stores a **complete snapshot** of your project at that moment (internally it deduplicates unchanged files, but conceptually: a full snapshot). Each snapshot is called a **commit**, and each commit knows which commit came before it — its **parent**.

So your project history is a **chain of snapshots**:

A —> B —> C —> D

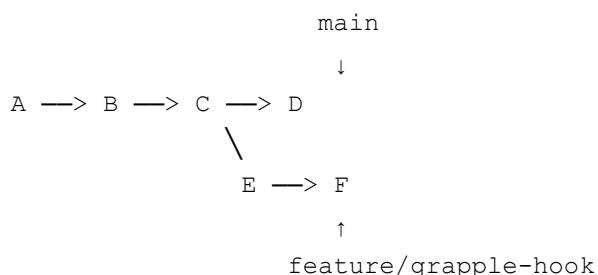
A commit contains:

- a snapshot of every tracked file
- the author, date, and message
- a pointer to its parent commit (or two parents, if it's a merge)
- a **hash** — a 40-character ID like `a3f9c81...` derived from all of the above

That hash is why git is trustworthy. Change anything in a commit — one byte of a texture, the date, the message — and the hash changes. Because each commit includes its parent's hash, changing an old commit changes every commit after it. History is tamper-evident by design. (This is also *why* rewriting history is disruptive — more on that in Part 6.)

Branches are sticky notes, not copies

A **branch** is a 41-byte file containing a commit hash. That's it. It's a pointer that moves forward when you commit.



Creating a branch does not copy your project. It writes one file. This is why git branching is instant even in a 40 GB game repo, and why you should branch constantly.

HEAD is another pointer: it points at the branch you're currently on. "Checking out" a branch means moving HEAD, then making your working files match that commit.

Git vs GitHub

	What it is
Git	The version control program running on your machine. Works fully offline. Created by Linus Torvalds, 2005.
GitHub	A website that hosts git repositories and adds collaboration features: pull requests, issues, code review, CI, releases. Owned by Microsoft.

GitHub is one of many hosts — GitLab, Bitbucket, Codeberg, and self-hosted options do the same thing. Git doesn't need any of them. When you `push`, you're copying commits from your machine to a computer somewhere else that also runs git.

Practical consequence: almost everything you do is local and instant. Only `clone`, `fetch`, `pull`, and `push` touch the network.

The one habit that makes git make sense

Run `git status` constantly. Before every command, after every command. It tells you what branch you're on, what's staged, what's modified, and — crucially — it tells you what to do next when you're stuck. Git's error messages and status output are unusually good at suggesting the exact command you need.

Part 2 — Setup and your first repo

One-time configuration

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Use the same email as your GitHub account so commits link to your profile.

Recommended settings, with reasons:

```
# Default branch name for new repos
git config --global init.defaultBranch main

# Show a common-ancestor section in conflicts — far easier to resolve
git config --global merge.conflictStyle zdiff3

# Don't create a merge commit every time you pull; replay your work on top
git config --global pull.rebase true

# When you push a new branch, auto-create it on the remote
git config --global push.autoSetupRemote true

# Remember conflict resolutions and reapply them automatically
git config --global rerere.enabled true

# Better diff algorithm for moved code blocks
git config --global diff.algorithm histogram
```

`rerere` ("reuse recorded resolution") is genuinely underrated: if you resolve the same conflict twice — which happens constantly on long-lived branches — git resolves it for you the second time.

Line endings (matters on mixed Windows/Mac teams):

```
# Windows
git config --global core.autocrlf true
# macOS / Linux
git config --global core.autocrlf input
```

Better still: commit a `.gitattributes` file with `* text=auto eol=lf` so the setting travels with the repo instead of depending on each teammate's config.

Check everything:

```
git config --list --show-origin
```

Authentication with GitHub

Passwords haven't worked since 2021. Two options:

SSH keys (recommended):

```
ssh-keygen -t ed25519 -C "you@example.com"
cat ~/.ssh/id_ed25519.pub          # copy this
# paste into GitHub → Settings → SSH and GPG keys → New SSH key
ssh -T git@github.com              # test it
```

GitHub CLI (easiest, and useful later for PRs):

```
gh auth login
```

Starting a repo

Two paths. Existing local project:

```
cd MyGame
git init
git add .
git commit -m "Initial commit"
git remote add origin git@github.com:you/MyGame.git
git push -u origin main
```

Or clone something that already exists:

```
git clone git@github.com:you/MyGame.git
cd MyGame
```

 **Game dev warning:** before your first `git add .`, write your `.gitignore`. Committing Unity's `Library/` or Unreal's `Intermediate/` folder into history is one of the most annoying mistakes to undo, and it will follow the repo forever. See Part 8.

.gitignore basics

A plain text file at the repo root listing paths git should pretend don't exist.

```
# Comment
Library/           # folder anywhere in the tree
/Builds/           # only at repo root
*.log              # any file with this extension
!important.log     # exception — do track this one
Temp/**/*.cache   # ** matches nested directories
```

Get a ready-made one for your engine: <https://github.com/github/gitignore>

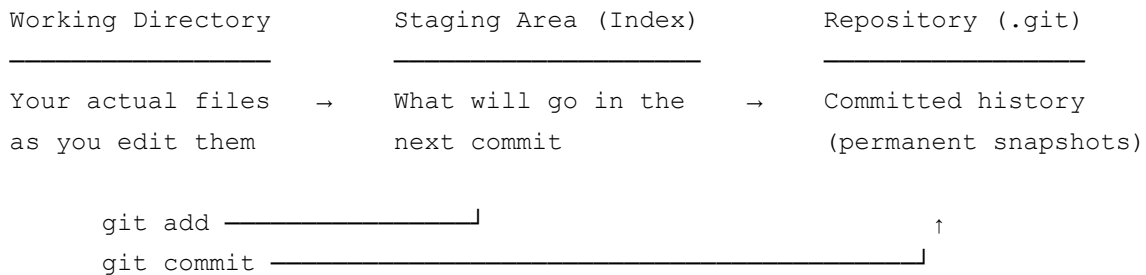
Key gotcha: `.gitignore` only affects **untracked** files. If a file is already committed, ignoring it does nothing. You must untrack it first:

```
git rm -r --cached Library/      # remove from git, KEEP on disk
git commit -m "Stop tracking Library"
```

Without `--cached` you'd delete it from disk too.

Part 3 — The three trees and the core loop

This is the concept that unlocks `add`, `reset`, `restore`, and `stash`. Git has three areas:



- **Working directory** — the files you see in your editor. Edit freely; git doesn't care yet.
- **Staging area (index)** — a rehearsal space. You choose which changes go into the next commit.
- **Repository** — the committed snapshots. Effectively permanent.

Most tutorials treat staging as bureaucracy. It's actually the feature that lets you commit *one logical change at a time* even when you've touched twelve files in a coding session.

The daily loop

```
git status          # what changed?
git diff            # exactly what changed (unstaged)
git add PlayerController.cs    # stage one file
git add Scripts/      # stage a folder
git add -p           # stage hunk-by-hunk (see below)
git diff --staged     # review what you're about to commit
git commit -m "Fix double-jump input buffer"
git log --oneline -10 # confirm
```

`git add -p` — the one power move to learn early

Interactive staging. Git walks you through each chunk of change and asks whether to stage it.

```
git add -p
```

Responses: `y` (stage it), `n` (skip), `s` (split into smaller chunks), `q` (quit), `e` (edit manually), `?` (help).

Real use: you fixed a bug in the enemy AI but also left three `Debug.Log` lines and renamed a variable. Use `-p` to stage only the bugfix, commit it cleanly, then handle the rest separately.

Writing commit messages that help future-you

The convention that most projects use:

```
Short summary in imperative mood, under ~50 chars
```

```
Optional body explaining WHY, not what. The diff already  
shows what changed. Explain the reasoning, the alternative  
you rejected, the bug number, the weird constraint.
```

```
Fixes #42
```

Imperative mood ("Add enemy spawner", not "Added" or "Adds") because git's own generated messages use it: *"Merge branch..."*, *"Revert..."*.

For a game project, useful prefixes are cheap and pay off when you're scanning 400 commits looking for when the jump felt wrong:

```
feat(player): add coyote time to jump  
fix(audio): stop footsteps looping after death  
art: swap placeholder cabin door texture  
balance: reduce shotgun spread by 15%  
chore: bump input system to 1.7.0
```

Reading history

```
git log                # full  
git log --oneline      # compact  
git log --oneline --graph --all  # visual branch structure — learn this one
```

```

git log -5                                # last 5
git log --author="Sam"
git log --since="2 weeks ago"
git log -- Assets/Scripts/Player.cs # history of one file
git log -p -- Player.cs                # ...with the actual diffs
git log -S "coyoteTime"                # commits that added/removed this string ("pi
git show a3f9c81                        # everything about one commit
git show a3f9c81:Assets/Player.cs      # a file as it existed at that commit

```

`git log -S` is the search you'll reach for when you think *"we deleted that function at some point, where did it go?"*

Make a nicer alias:

```

git config --global alias.lg "log --oneline --graph --decorate --all -20"
# now: git lg

```

Diffing

```

git diff                                # working dir vs staging
git diff --staged                       # staging vs last commit
git diff HEAD                           # working dir vs last commit
git diff main..feature/enemies         # between branches
git diff HEAD~3 HEAD                   # last three commits' combined change
git diff --stat                         # summary: files changed, +/- counts
git diff --name-only main               # just the filenames

```

Referring to commits

You'll see these everywhere:

Reference	Means
HEAD	The commit you're currently on
HEAD~1 or HEAD^	Its parent (one commit back)
HEAD~3	Three commits back
a3f9c81	A specific commit by hash (first 7 chars is usually enough)
main	The tip of main

origin/main	The tip of main <i>as of your last fetch</i>
v1.2.0	A tag
HEAD@{2}	Where HEAD was two moves ago (see reflog, Part 6)

Part 4 — Branches: the real explanation

Why branch

Because you want to build the grappling hook without breaking the build your artist is using to check lighting. Branches isolate work in progress.

Commands

```
git branch                # list local branches
git branch -a             # include remote-tracking branches
git branch feature/grapple # create (but stay where you are)
git switch feature/grapple # move onto it
git switch -c feature/grapple # create AND move (most common)
git switch main           # go back
git switch -              # go back to previous branch (like cd -)
git branch -d feature/grapple # delete (safe: refuses if unmerged)
git branch -D feature/grapple # force delete
git branch -m old-name new-name # rename
```

`switch` and `restore` (git 2.23+) replaced the overloaded `checkout`, which did both jobs and confused everyone. `git checkout` still works and you'll see it in older tutorials. Prefer `switch` for branches, `restore` for files.

What actually happens

Start with three commits on `main`:

```
A —> B —> C    ← main ← HEAD
```

```
git switch -c feature/grapple:
```

```
A —> B —> C    ← main
```

```
← feature/grapple ← HEAD
```

Nothing copied. Two pointers at the same commit. Now commit twice:

```
A → B → C          ← main
      |
      D → E          ← feature/grapple ← HEAD
```

Switch back to main and your files revert to state C. The grapple code isn't gone — it's in commits D and E, reachable through the `feature/grapple` pointer.

Naming conventions

Pick one and stick to it:

```
feature/grapple-hook
fix/audio-desync
art/cabin-interior-pass2
exp/ragdoll-test      # experimental, may be thrown away
release/v0.4
```

Slashes are just characters in a branch name, but most tools display them as folders — nice when you have 30 branches.

Detached HEAD

If you check out a *commit* rather than a branch:

```
git switch --detach a3f9c81
# or: git checkout a3f9c81
```

HEAD now points directly at a commit rather than a branch. You can look around, build, test. But **commits you make here belong to no branch** — when you switch away, they become unreachable and eventually get garbage collected.

Git warns you loudly. If you made changes worth keeping:

```
git switch -c my-rescued-work    # creates a branch here, work is safe
```

If you already switched away and panicked, see `git reflog` in Part 6. Nothing is lost for ~90 days.

Legitimate use: checking out an old tag to build a previous version, or testing what the game felt like fifteen commits ago.

Part 5 — Remotes, fetch, pull, push, merge, conflicts

Remotes

A **remote** is a nickname for a repository URL. `origin` is the conventional name for "the one I cloned from."

```
git remote -v                # list remotes
git remote add origin git@github.com:you/Game.git
git remote add upstream git@github.com:studio/Game.git # common in forks
git remote set-url origin git@github.com:you/NewName.git
git remote remove old-thing
```

Remote-tracking branches — the piece everyone misses

When you clone, git creates two kinds of branch:

- `main` — **your** local branch, which you edit
- `origin/main` — a **read-only bookmark** recording where `main` was on the server *the last time you talked to it*

`origin/main` does not update magically. It updates only when you `fetch`, `pull`, or `push`. So if `git log` shows `origin/main` behind, that may just mean your information is stale, not that the server is behind.

Your machine		GitHub
main	(your work)	
origin/main	(last known server state)	↔ main (actual server state)

Fetch vs pull — the distinction that prevents most confusion

```
git fetch origin            # download new commits, update origin/*, TOUCH NOTHING EL
git pull origin main        # fetch + integrate into your current branch
```

`fetch` is always safe. It can never cause a conflict or change your files. It just updates your knowledge.

`pull` is `fetch` followed by either `merge` or `rebase`. It *can* cause conflicts, because it changes your working files.

A better daily habit than blind pulling:

```
git fetch origin
git log --oneline HEAD..origin/main    # what did others do that I don't have?
git diff HEAD origin/main --stat      # which files will change?
git pull                               # now integrate, knowing what's coming
```

That three-second check is the difference between "pull and pray" and knowing that someone rewrote the scene file you've been editing all morning.

pull --rebase vs pull --merge

```
git pull --rebase    # replay YOUR commits on top of theirs → linear history
git pull --no-rebase # create a merge commit → preserves exact history
```

If you set `pull.rebase true` in Part 2, rebase is your default. It avoids the clutter of "Merge branch 'main' of github.com:..." commits that add no information. Use `--no-rebase` when you've already shared your commits with someone else (see the Golden Rule in Part 6).

Push

```
git push                                # push current branch to its tracked remote
git push -u origin feature/grapple     # first push of a new branch; -u sets trackin
git push origin --delete old-branch    # delete a branch on the server
git push --tags                         # push tags (they don't go automatically)
```

`-u ( --set-upstream )` links your local branch to a remote one so future `git push` and `git pull` need no arguments.

Push only fast-forwards. Git will reject a push that would make the server *lose* commits. This is the single most common error new users hit:

```
! [rejected]  main -> main (fetch first)
error: failed to push some refs
hint: Updates were rejected because the remote contains work that you do not have
```

Translation: someone pushed while you were working. Fix:

```
git pull --rebase
# resolve conflicts if any
git push
```

Merging

Merging combines another branch's work into your current branch.

```
git switch main
git merge feature/grapple
```

Case 1: fast-forward

If main hasn't moved since you branched, git just slides the pointer forward. No merge commit, no conflict possible.

```
Before:  A —> B —> C — main
              \
              D —> E — feature

After:   A —> B —> C —> D —> E — main, feature
```

Case 2: three-way merge

If both branches have new commits, git finds the common ancestor (C), compares both sides against it, and creates a **merge commit** with two parents:

```
Before:  A —> B —> C —> F —> G — main
              \
              D —> E — feature

After:   A —> B —> C —> F —> G —> M — main
              \               /
              D —> E —————
```

Useful flags:

```
git merge --no-ff feature/grapple # force a merge commit even if FF possible
                                   # (keeps the feature visible as a unit)
git merge --squash feature/grapple # collapse all its commits into one staged ch
```

```
git merge --abort                                # bail out mid-conflict, back to before
```

`--squash` is great for a messy 23-commit branch full of "wip", "wip2", "actually fix it".

Conflicts

What causes one

A conflict happens when **both branches changed the same region of the same file** relative to the common ancestor. That's it. Not "both touched the same file" — git merges different regions of a file fine.

Git *cannot* auto-merge:

- overlapping edits to the same lines
- one side edited a file, the other deleted it
- both sides added a file with the same name but different content
- **any change to a binary file** — git can't merge two versions of a `.png` or `.uasset`, so it always conflicts

Anatomy of a conflict

```
<<<<<<< HEAD
    jumpForce = 12.5f;
||||||| merged common ancestor
    jumpForce = 10f;
=====
    jumpForce = 8f;
>>>>>>> feature/floaty-jump
```

- Between `<<<<<<<` and `|||||||` — **your** version (the branch you're on)
- Between `|||||||` and `=====` — the **original** both of you started from (only shown with `merge.conflictStyle = zdiff3`, which you set in Part 2 — turn it on, it turns guesswork into information)
- Between `=====` and `>>>>>>>` — **their** version (the branch being merged in)

Here the ancestor tells you the whole story: you raised it, they lowered it. That's a design disagreement, not a mechanical merge problem — go ask which one is right.

The resolution workflow

```

git merge feature/floaty-jump
# CONFLICT (content): Merge conflict in Assets/Scripts/Player.cs
# Automatic merge failed; fix conflicts and then commit the result.

git status          # lists files "both modified"
# 1. Open each file. Find every <<<<<<< marker.
# 2. Edit until the file says what you want. DELETE all markers.
# 3. Actually run the game – a syntactically valid merge can still be logically b

git add Assets/Scripts/Player.cs  # "add" here means "I resolved this"
git status                        # repeat until nothing is unmerged
git commit                        # git pre-writes the merge message

```

Escape hatches and shortcuts:

```

git merge --abort          # undo everything, back to pre-merge state
git checkout --ours Player.cs  # take MY version of this file entirely
git checkout --theirs Player.cs  # take THEIR version entirely
git diff --name-only --diff-filter=U  # list only unresolved files
git mergetool              # open your configured 3-way merge GUI

```

⚠ Big gotcha: `--ours` / `--theirs` mean opposite things during a **rebase**. In a merge, "ours" = the branch you're on. In a rebase, git replays your commits onto the other branch, so "ours" = the branch you're rebasing *onto* and "theirs" = *your* commits. When in doubt during a rebase, check `git status` — it tells you which operation you're in.

Making conflicts rarer

- Pull/rebase from main daily. A one-day-old branch conflicts on a few lines; a three-week-old branch conflicts on everything.
- Keep branches small and short-lived.
- Split monolithic files. One 4,000-line `GameManager.cs` guarantees conflicts; ten focused files rarely collide.
- Agree on formatting up front (`.editorconfig` , a formatter in CI). Half of all "conflicts" are whitespace and brace-style wars.
- For scenes, prefabs, and other binary-ish assets: **coordinate, don't merge** (Part 8).

Part 6 — Rebase, history rewriting, and undo

Rebase

Merge preserves the exact shape of what happened. Rebase rewrites your work so it *looks like* you started from the latest main.

```
git switch feature/grapple
git rebase main
```

Before: A —> B —> C —> F —> G — main
 \
 D —> E — feature

After: A —> B —> C —> F —> G — main
 \
 D' —> E' — feature

D' and E' are **new commits** with new hashes — same changes, different parents. The originals still exist temporarily but nothing points at them.

Rebase conflicts happen per-commit, so you resolve, then continue:

```
# resolve files, then:
git add <file>
git rebase --continue
# or
git rebase --skip        # drop this commit
git rebase --abort      # give up entirely, nothing lost
```

The Golden Rule

Never rebase commits that other people have based work on.

Rewriting shared history means your teammates' repos now disagree with the server about what happened, and the cleanup is genuinely painful. Rule of thumb: rebase your *own* unpushed or unshared branch freely. Never rebase `main`.

Merge or rebase?

Use merge when	Use rebase when
Integrating a feature branch into main	Updating your branch with new main before a PR
The branch is shared	The branch is yours alone

You want the true history

You want a clean, linear, reviewable history

Many teams settle on: **rebase to update your branch, merge to land it.**

Interactive rebase — cleaning up before a PR

```
git rebase -i HEAD~5      # edit the last 5 commits
git rebase -i main        # edit everything since main
```

An editor opens:

```
pick a1b2c3d Add grapple raycast
pick d4e5f6a wip
pick 7890abc fix typo
pick bcd1234 actually fix the rope physics
pick ef56789 remove debug logs
```

Change the first word of each line:

Command	Effect
pick	keep as-is
reword	keep the change, edit the message
edit	pause here so you can amend the commit
squash	merge into the previous commit, combine messages
fixup	merge into the previous commit, discard this message
drop	delete the commit entirely

You can also reorder or delete lines. Save and close; git replays accordingly.

Turning that mess into two clean commits:

```
pick a1b2c3d Add grapple raycast
fixup d4e5f6a
fixup 7890abc
pick bcd1234 Fix rope physics oscillation
fixup ef56789
```

Reviewers will thank you.

Amend — fix the last commit

```
git commit --amend -m "Better message"      # change the message
git add forgotten-file.cs
git commit --amend --no-edit                 # add a file, keep the message
```

Amend creates a *new* commit replacing the old one. Safe if unpushed; if pushed, you'll need `--force-with-lease`.

The undo table

This is the section to bookmark. "Undo" means five different things in git:

Situation	Command	What it does
Discard unstaged edits to a file	<code>git restore Player.cs</code>	Reverts file to last commit. Destroys work permanently.
Discard all unstaged edits	<code>git restore .</code>	Same, everywhere. Careful.
Unstage a file (keep edits)	<code>git restore --staged Player.cs</code>	Moves it out of staging
Undo last commit, keep changes staged	<code>git reset --soft HEAD~1</code>	Commit gone, work still staged
Undo last commit, keep changes unstaged	<code>git reset HEAD~1</code>	Default <code>--mixed</code>
Undo last commit, obliterate changes	<code>git reset --hard HEAD~1</code>	Destroys work.
Undo a commit that's already pushed	<code>git revert a3f9c81</code>	Creates a NEW commit undoing it. Safe for shared history.
Recover something you "lost"	<code>git reflog</code>	See below

reset vs revert — the important distinction

- `reset` moves the branch pointer backwards. History is rewritten. Fine for local,

unshared commits. Dangerous on pushed branches.

- **revert** adds a new commit that applies the inverse change. History is preserved. **This is what you use on main.**

```
git revert a3f9c81          # undo one commit
git revert HEAD~3..HEAD     # undo a range
git revert -m 1 <merge-commit> # undo a merge (-m 1 = keep the main-line parent)
```

git reflog — your safety net

Git records every movement of HEAD: every commit, checkout, reset, rebase, merge. Even "lost" commits stay reachable for ~90 days by default.

```
git reflog
```

```
a3f9c81 HEAD@{0}: reset: moving to HEAD~3
9d8c7b6 HEAD@{1}: commit: Add boss phase 2
5f4e3d2 HEAD@{2}: commit: Add boss phase 1
```

Deleted three commits by accident? They're right there:

```
git reset --hard HEAD@{1}      # back to before the reset
# or rescue them onto a new branch:
git switch -c rescue 9d8c7b6
```

Almost nothing is truly lost in git if it was ever committed. Committing often is what makes this true — uncommitted work has no safety net at all.

Stash — park work temporarily

```
git stash          # shelve all uncommitted changes
git stash -u       # include untracked files (important for new a
git stash push -m "grapple wip" # named stash
git stash list     # see them all
git stash pop      # apply most recent AND remove from stash
git stash apply stash@{2} # apply a specific one, keep it in the list
git stash drop stash@{0}
git stash show -p stash@{0} # preview the diff
```

Plain `git stash` ignores untracked files — a classic way to "lose" a brand-new script or

texture. Use `-u`.

Honestly, for anything you'll return to later than this afternoon, a WIP commit on a scratch branch is safer and more visible than a stash.

Cherry-pick — grab one specific commit

```
git cherry-pick a3f9c81          # apply that commit here
git cherry-pick a3f9c81 d4e5f6a  # several
git cherry-pick a3f9c81..bcd1234 # a range
git cherry-pick --no-commit a3f9c81 # apply changes but don't commit yet
```

Classic use: you fixed a crash on your feature branch, and the demo build going out tomorrow needs *just that fix*, not the rest of your half-finished work.

Force pushing safely

After a rebase or amend, your local history no longer matches the remote, so a normal push is rejected.

```
git push --force-with-lease  #  use this
git push --force             #  avoid
```

`--force-with-lease` refuses if someone else pushed since your last fetch — it protects your teammates' work. Plain `--force` will happily delete it.

Tags — marking builds

```
git tag -a v0.3.0 -m "Vertical slice for publisher demo"
git tag                                # list
git push origin v0.3.0                 # tags do NOT push automatically
git push --tags                       # push all
git tag -d v0.3.0                     # delete locally
git push origin --delete v0.3.0       # delete on remote
git switch --detach v0.3.0            # check out that exact build state
```

Tag every build you send anyone — publisher, playtester, storefront. When a tester reports a bug in "the July build", you can check out exactly that code.

Bisect — find the commit that broke it

Binary search through history. Enormously useful for "the jump felt fine last month."

```
git bisect start
git bisect bad           # current state is broken
git bisect good v0.2.0   # this old tag was fine
# git checks out a commit halfway between; you test the game
git bisect good          # or: git bisect bad
# repeat ~log2(N) times
git bisect reset         # done — returns you to where you started
```

With 1,000 commits between good and bad, that's about 10 tests. Automate it if you have a test script:

```
git bisect run ./run-tests.sh
```

Worktrees — two branches checked out at once

```
git worktree add ../Game-hotfix main
cd ../Game-hotfix
```

A second working directory sharing one `.git`. For game devs this is a real quality-of-life win: switching branches in a big Unity/Unreal project triggers a long asset reimport or shader recompile. With a worktree, you keep your feature branch open in one folder and fix the hotfix in another, no reimport churn.

```
git worktree list
git worktree remove ../Game-hotfix
```

(Each worktree gets its own `Library/ / Intermediate/` cache, so the first open there is slow — but only once.)

Part 7 — GitHub: PRs, forks, reviews, automation

The pull request

A PR is a request to merge one branch into another, wrapped in a discussion and review UI. Nothing about it is part of git itself — it's GitHub's layer on top of `git merge`.

Typical lifecycle:

```
git switch main
git pull
git switch -c feature/enemy-patrol
# ...work, commit...
git push -u origin feature/enemy-patrol
```

GitHub prints a link; or use the CLI:

```
gh pr create --title "Add enemy patrol AI" --body "Closes #42"
gh pr list
gh pr checkout 57          # check out someone else's PR locally to playtest it
gh pr view 57 --web
gh pr merge 57 --squash --delete-branch
```

Writing a PR description that gets reviewed quickly:

```
## What
Adds patrol behaviour to the basic enemy: waypoint following,
idle pause, and a 4s search state when the player breaks line of sight.

## Why
Closes #42. Playtesters said enemies felt static and predictable.

## How to test
1. Open Scenes/Test_Patrol.unity
2. Play. Enemy should walk between the four markers.
3. Walk into its cone, then break LoS – it should search, not snap back.

## Notes
- Search timing is exposed on the ScriptableObject, currently 4s, likely needs tu
- Does not touch the boss AI, which still uses the old state machine
```

Keep PRs small. A 200-line PR gets a real review; a 4,000-line PR gets "LGTM 👍" and ships bugs.

The three merge buttons

Button	Result	When
Create a merge commit	Preserves every commit + a merge commit	You want full history
Squash and merge	All commits collapse into one on main	Default for most teams — main stays readable

Rebase and merge	Commits replayed onto main, no merge commit	You want linear history and each commit is meaningful
-------------------------	---	---

Squash-merge is the easy default: developers commit as messily as they like on their branch, main stays clean.

Review mechanics

- **Comment** — feedback, no verdict
- **Approve** — good to merge
- **Request changes** — blocks merging until resolved
- **Suggested changes** — propose exact replacement code the author can commit with one click

Draft PRs (`gh pr create --draft`) are underused: open one on day one so teammates can see direction early without being asked to review unfinished work.

Forks

A fork is your own server-side copy of someone else's repo. You need one when you don't have write access — contributing to an open-source engine plugin, for example.

```
gh repo fork studio/CoolPlugin --clone
cd CoolPlugin
git remote -v
# origin    → your fork
# upstream  → the original
```

Staying in sync:

```
git fetch upstream
git switch main
git merge upstream/main      # or: git rebase upstream/main
git push origin main
```

Then branch, push to *your* fork, and open a PR against the original.

Repo hygiene worth setting up

- **Branch protection** on `main` : require a PR, require review, require CI to pass, block force

pushes. Settings → Branches → Add rule.

- **CODEOWNERS** — auto-request reviews from the right person:

```
# .github/CODEOWNERS
/Assets/Art/           @your-artist
/Assets/Scripts/AI/     @gameplay-dev
/Assets/Audio/          @sound-designer
*.cs                    @you
```

- **Issue and PR templates** in `.github/`
- **Releases** — attach build artifacts to a tag so playtesters download from a stable link instead of Discord uploads

GitHub Actions in one page

CI: run something automatically on push/PR. Config lives in `.github/workflows/*.yaml`.

```
name: Build check
on:
  pull_request:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: true
      - name: Run tests
        run: ./run-tests.sh
```

Useful for a game project even at small scale:

- verify the project still compiles before a merge lands
- run unit tests on your pure-logic code
- lint / format check so formatting never causes a conflict again
- nightly builds uploaded as artifacts

⚠ Full engine builds on hosted runners are slow and, for Unity, need license activation. Many small teams run CI on a self-hosted runner (an old PC in the corner) instead. Start with "does it compile", not "build every platform".

Workflow models

Model	Shape	Fits
Solo / trunk	Commit straight to main, branch only for risky work	One person, prototype
Feature branch	Branch per feature → PR → main. Main always shippable	Most small teams. Start here.
Git Flow	main + develop + feature/release/hotfix branches	Scheduled releases, versioned products. Heavy for small teams
Trunk-based	Tiny branches merged daily, features hidden behind flags	Fast-moving teams with good CI
Forking	Everyone works in a personal fork	Open source, untrusted contributors

For a small game team, feature-branch + squash-merge + protected main covers 95% of needs.

Part 8 — Game dev specifics

This is where game development departs sharply from typical software use of git. Git was designed for text. Games are mostly not text.

The three problems

1. **Binary files can't be merged.** Two people edit the same texture, mesh, scene, or prefab, and git can only offer you "take mine or take theirs." There is no middle ground.
2. **Binary files bloat history.** Every version of every 40 MB texture is stored forever. Repos hit tens of gigabytes and clones take an hour.
3. **Engines generate huge caches** that must never be committed.

Solutions, in order: `.gitignore` → Git LFS → file locking → engine-specific merge tools.

Git LFS (Large File Storage)

LFS replaces large files in your repo with tiny text pointers, storing the actual bytes on a separate server. Your repo history stays small; you only download the versions of assets you actually check out.

```
git lfs install                                # one-time, per machine

git lfs track "*.psd"
git lfs track "*.png"
git lfs track "*.fbx"
git lfs track "*.wav"
git lfs track "*.uasset"
git lfs track "*.umap"

git add .gitattributes                        # ← this file MUST be committed
git commit -m "Track binary assets with LFS"
```

`.gitattributes` ends up looking like:

```
*.psd  filter=lfs diff=lfs merge=lfs -text
*.fbx  filter=lfs diff=lfs merge=lfs -text
*.png  filter=lfs diff=lfs merge=lfs -text
```

Useful commands:

```
git lfs ls-files                            # what's in LFS
git lfs status
git lfs pull                                # fetch LFS content for current checkout
git lfs migrate info                        # analyse repo: what's big, what should move
```

Set LFS up on day one. Adding it later means rewriting history (there's a scenario for that in Part 9).

⚠️ GitHub's free LFS quota is small (1 GB storage / 1 GB bandwidth per month by default; paid data packs beyond that). A texture-heavy project burns through it fast. Check current pricing before committing to GitHub as your asset host — some studios use GitHub for code and a separate solution (self-hosted LFS, Perforce, or a cloud drive) for heavy art.

File locking — the actual answer for binary conflicts

You cannot merge two versions of a scene. So don't create two versions. LFS supports locking: claim a file, others can't edit it until you release it.

Mark file types as lockable in `.gitattributes` :

```
*.uasset  filter=lfs diff=lfs merge=lfs -text lockable
*.umap    filter=lfs diff=lfs merge=lfs -text lockable
*.unity   lockable
*.prefab  lockable
```

Then:

```
git lfs lock Content/Maps/Level_01.umap
git lfs locks # who has what
git lfs unlock Content/Maps/Level_01.umap
git lfs unlock --force <file> # admin override when someone's on holiday
```

With `lockable` , those files are read-only on disk until you lock them — which stops the "oops, I edited it for two hours before noticing" problem at the source.

Social version of the same rule: in a small team, a pinned Discord/Slack message saying "I'm in the cabin level" works nearly as well. What matters is that *someone owns each binary file at a time*.

Unity setup

1. Project Settings → Editor:

- *Asset Serialization* → Mode: **Force Text** — makes scenes and prefabs YAML instead of binary, so diffs are readable and small merges become possible
- *Version Control* → Mode: **Visible Meta Files**

2. Always commit `.meta` files. Unity stores GUIDs, import settings, and references in them. A missing `.meta` means broken references and reimported assets for everyone. Never gitignore them.

3. `.gitignore` :

```
[Ll]ibrary/
[Tt]emp/
[Oo]bj/
[Bb]uild/
[Bb]uilds/
[Ll]ogs/
[Uu]serSettings/
[Mm]emoryCaptures/
```

```
.vs/  
.idea/  
*.csproj  
*.sln  
*.unitypackage  
sysinfo.txt
```

Grab the maintained version: <https://github.com/github/gitignore/blob/main/Unity.gitignore>

4. UnityYAMLMerge (Smart Merge) — Unity ships a merge tool that understands scene/prefab YAML and can merge many changes automatically. Configure it in your global

`.gitconfig`:

```
[merge]  
  tool = unityyamlmerge  
[mergetool "unityyamlmerge"]  
  trustExitCode = false  
  cmd = '/Applications/Unity/Hub/Editor/2022.3.20f1/Unity.app/Contents/Tools/Un
```

(Windows path is typically `C:\Program Files\Unity\Hub\Editor\
<version>\Editor\Data\Tools\UnityYAMLMerge.exe`.)

And in `.gitattributes`:

```
*.unity    merge=unityyamlmerge eol=lf  
*.prefab   merge=unityyamlmerge eol=lf  
*.asset    merge=unityyamlmerge eol=lf
```

Then `git mergetool` on a conflicted scene. It resolves many cases — but **always open the merged scene in Unity and look at it** before committing. A YAML-valid scene can still be a scene where the boss arena has two floors and no lights.

5. Structural habits that prevent scene conflicts:

- Break levels into additive sub-scenes, one per person
- Put logic in prefabs, not loose scene objects — prefab edits conflict far less than scene edits
- Use nested prefabs and prefab variants
- Keep one big shared scene as small as possible; it's the highest-conflict file in most Unity projects

Unreal setup

1. .gitignore :

```
Binaries/  
Build/  
DerivedDataCache/  
Intermediate/  
Saved/  
.vs/  
*.sln  
*.suo  
*.opensdf  
*.sdf
```

Keep: Config/ , Content/ , Source/ , and the .uproject .

2. LFS everything binary — .uasset and .umap are the whole project:

```
*.uasset filter=lfs diff=lfs merge=lfs -text lockable  
*.umap   filter=lfs diff=lfs merge=lfs -text lockable
```

3. Turn on One File Per Actor (UE5). Instead of one giant .umap that everyone fights over, each actor gets its own file. Two people editing different parts of the same level stop conflicting entirely. This is the single biggest Unreal-on-git improvement available.

4. Use World Partition for large levels — same benefit, spatially.

5. Be honest about the tradeoff: Unreal's own tooling is built around Perforce, and studios with lots of artists often end up there. Git + LFS + OFPA + locking is genuinely workable for small teams; at 15 artists it starts to hurt.

Godot setup

Godot is the friendliest of the three for git — scenes (.tscn) and resources (.tres) are text and diff readably.

```
.godot/           # Godot 4  
.import/          # Godot 3  
export_presets.cfg # often contains keystore paths/passwords  
builds/  
*.translation
```

Still LFS your `.png`, `.wav`, `.ogg`, `.blend`.

`.tscn` conflicts are real conflicts you can actually read and resolve by hand — but be careful with node IDs and `ext_resource` numbering; mismatched IDs silently break references. Open the scene in the editor after every manual resolution.

Keeping the repo small

```
git count-objects -vH          # how big is my repo really?
git lfs migrate info           # which file types dominate history?
```

Rules that keep a game repo healthy for years:

- Never commit builds, packaged output, or crash dumps
- Never commit the engine cache directories
- LFS from day one
- Source art (`.psd`, `.blend`, 4K source textures) — consider a separate repo or cloud drive; game repos only need the exported assets
- Don't commit huge video files; link them

For enormous existing repos:

```
git clone --filter=blob:none <url>          # partial clone: fetch blobs on demand
git sparse-checkout set Assets/Scripts      # only check out part of the tree
```

Submodules — shared code between projects

If you have a shared tools/library repo used by several games:

```
git submodule add git@github.com:you/GameTools.git Assets/Plugins/GameTools
git commit -m "Add GameTools submodule"

# for people cloning:
git clone --recurse-submodules <url>
# or after the fact:
git submodule update --init --recursive

# update to the latest upstream commit:
cd Assets/Plugins/GameTools && git pull origin main
cd ../../.. && git add Assets/Plugins/GameTools && git commit -m "Bump GameTools"
```

Submodules pin an exact commit of the other repo — that's their value and their annoyance. Teammates constantly forget `--recurse-submodules` and then wonder why the folder is empty. If your engine has a package manager (Unity Package Manager can install from a git URL), that's usually a smoother option.

Part 9 — Scenario library

Realistic situations with the exact commands. Skim the headings; come back when one happens to you.

1. Standard day: build a feature and ship it

You're adding a grappling hook.

```
git switch main
git pull                                # start from current main

git switch -c feature/grapple-hook
# ...work...
git add Assets/Scripts/Grapple/
git commit -m "feat(player): add grapple raycast and rope render"
# ...more work...
git commit -am "feat(player): add rope swing physics"

git fetch origin                        # what's happened on main today?
git rebase origin/main                 # replay my work on top of it
git push -u origin feature/grapple-hook
gh pr create
```

Review comes back, you make changes:

```
git commit -am "fix: clamp rope length to max distance"
git push
```

Merged. Clean up:

```
git switch main
git pull
git branch -d feature/grapple-hook
git push origin --delete feature/grapple-hook
```

2. "Updates were rejected" when pushing

```
! [rejected] main -> main (fetch first)
```

Someone pushed while you were working. Never force here.

```
git pull --rebase
# resolve conflicts if any, then:
git rebase --continue
git push
```

3. Two people changed the same C# file

You and a teammate both edited `PlayerController.cs`.

```
git pull --rebase
# CONFLICT (content): Merge conflict in Assets/Scripts/PlayerController.cs
```

Open it:

```
<<<<<< HEAD
    private float jumpForce = 12.5f;
    private float coyoteTime = 0.15f;
|||||| parent of 8a3f1c2
    private float jumpForce = 10f;
=====
    private float jumpForce = 10f;
    private float jumpBufferTime = 0.1f;
>>>>>> 8a3f1c2 (Add jump buffering)
```

The ancestor line shows what happened: you changed `jumpForce` and added `coyoteTime`; they kept `jumpForce` and added `buffering`. Both features should survive:

```
private float jumpForce = 12.5f;
private float coyoteTime = 0.15f;
private float jumpBufferTime = 0.1f;
```

```
git add Assets/Scripts/PlayerController.cs
```

```
git rebase --continue
# Now actually play the game and jump. Compiling ≠ correct.
git push
```

4. Two people changed the same Unity scene

The hard one. Git shows a conflict in `Level_02.unity` and there's no sane way to hand-merge 8,000 lines of YAML.

Option A — one side wins (most common, fastest):

```
git checkout --ours Assets/Scenes/Level_02.unity    # keep mine
# or
git checkout --theirs Assets/Scenes/Level_02.unity  # keep theirs
git add Assets/Scenes/Level_02.unity
git merge --continue
```

Then whoever lost re-does their work by hand. Usually 20 minutes. Painful but predictable.

Option B — Smart Merge (if Force Text serialization is on):

```
git mergetool          # runs UnityYAMLMerge
# then OPEN THE SCENE IN UNITY AND VERIFY BEFORE COMMITTING
git add Assets/Scenes/Level_02.unity
git merge --continue
```

Option C — extract, reset, reapply:

```
# 1. Copy your version somewhere outside the repo
cp Assets/Scenes/Level_02.unity ~/Desktop/mine.unity
# 2. Take theirs
git checkout --theirs Assets/Scenes/Level_02.unity
git add . && git merge --continue
# 3. In Unity, open both scenes additively and copy your objects across as prefab
```

The real fix is prevention: `git lfs lock` the scene, or split it into additive sub-scenes so you and your teammate are never in the same file.

5. You committed to main by accident

Three commits deep before you notice you never branched.

```
git switch -c feature/enemy-ai      # branch here — your commits come with yo
git switch main
git reset --hard origin/main        # main back to what the server has
git switch feature/enemy-ai         # carry on
```

If you already **pushed** to main, don't reset. Use revert or move the work properly and coordinate with your team before touching shared history.

6. Urgent hotfix while you're mid-feature

The demo build crashes on launch and your working tree is a half-finished inventory system.

With a worktree (best for Unity/Unreal — no reimport):

```
git worktree add ../Game-hotfix main
cd ../Game-hotfix
git switch -c fix/null-save-crash
# fix, commit, push, PR
cd ../Game
git worktree remove ../Game-hotfix
```

With stash (simpler, but you pay the reimport):

```
git stash -u -m "inventory wip"
git switch main && git pull
git switch -c fix/null-save-crash
# fix, commit, push
git switch feature/inventory
git stash pop
```

7. You need one fix from another branch, not the whole branch

Your `feature/boss-fight` branch contains a null-check fix that the demo needs tonight.

```
git log --oneline feature/boss-fight # find the commit
git switch release/demo
git cherry-pick 7c3d9a1
git push
```

If the fix is tangled up with other changes in that commit, split it: cherry-pick with `--no-`

`commit` , unstage what you don't want, then commit.

8. A bad commit is already on main

Yesterday's "optimization" tanked the framerate and it's already merged.

```
git switch main
git pull
git revert a3f9c81          # creates "Revert: optimize particle pooling"
git push
```

Reverting a merge commit needs a parent number:

```
git revert -m 1 <merge-commit-hash>    # -m 1 = keep main's side
```

9. You deleted a branch that wasn't merged

```
git reflog                # find the last commit that was on it
# ...
# 9d8c7b6 HEAD@{14}: commit: Add boss phase 2
git switch -c feature/boss-rescued 9d8c7b6
```

Or if you know the hash from the delete warning git printed, use that directly.

10. `git reset --hard` ate two hours of work

If it was committed: completely recoverable.

```
git reflog
git reset --hard HEAD@{1}
```

If it was never committed: it's gone. Git had no record of it. Some editors keep local history (JetBrains "Local History", VS Code timeline) — check there. Then start committing more often; that's the actual lesson.

11. You committed a 3 GB build folder

Repo is now enormous and clones take forever.

If it's the **most recent commit and unpushed**:

```
git reset --soft HEAD~1
echo "Builds/" >> .gitignore
git restore --staged Builds/
git add .gitignore
git commit -m "Add build output to gitignore"
```

If it's **buried in history**:

```
pip install git-filter-repo
git clone --mirror git@github.com:you/Game.git Game-mirror
cd Game-mirror
git filter-repo --path Builds/ --invert-paths
git push --force
```

Then **every teammate must re-clone**. Every hash after the removed commit changed. Announce it loudly, pick a quiet moment, make sure everyone has pushed first.

12. You committed a secret (Steam key, API token, signing keystore)

```
# 1. ROTATE THE SECRET FIRST. Assume it is compromised the moment it was pushed.
#   Anyone can read it in the history, and GitHub's caches keep things around.

# 2. Then scrub history:
git filter-repo --path Config/steam_key.txt --invert-paths
git push --force

# 3. Prevent recurrence:
echo "Config/secrets/" >> .gitignore
```

Rotation is not optional and not the second step you skip. Scrubbing history without rotating leaves you exposed.

13. Adding LFS to a repo that's already fat

```
git lfs install
git lfs migrate info # see what's actually taking space
```

```
git lfs migrate import --everything \
  --include="*.psd,*.png,*.fbx,*.wav,*.tga"

git push --force --all
git push --force --tags
```

This rewrites history. Same protocol as scenario 11: everyone pushes, everyone re-clones, done on a Friday afternoon or during a lull.

14. Your branch is 40 commits behind main

```
git fetch origin
git rebase origin/main
```

Conflicts appear one commit at a time:

```
# resolve, then
git add <files>
git rebase --continue
# repeat
```

If it turns into a nightmare (10+ conflicting commits), abort and merge instead — one big conflict resolution is often easier than forty small ones:

```
git rebase --abort
git merge origin/main
```

And with `rerere.enabled`, repeated conflicts resolve themselves.

15. Clean up a messy branch before opening a PR

Your history: `wip`, `wip2`, `fix`, `oops`, actually works now.

```
git rebase -i main

pick alb2c3d Add enemy patrol state machine
fixup d4e5f6a wip
fixup 7890abc wip2
pick bcd1234 Add search behaviour on lost LoS
```

```
fixup ef56789 oops
fixup 1234567 actually works now
```

Save, close, then:

```
git push --force-with-lease
```

Only safe because nobody else is working on this branch.

16. Playtest which build broke the jump feel

Testers say jumping felt right in the July demo and wrong now.

```
git bisect start
git bisect bad                # today is bad
git bisect good v0.2.0-july-demo # the demo tag was good
# git checks out a middle commit — build, play, judge
git bisect good              # or bad
# repeat ~8 times
# git prints: "a3f9c81 is the first bad commit"
git bisect reset
git show a3f9c81
```

For feel-based bugs with no automated test, bisect is the only systematic tool you have.

17. Review a teammate's PR by actually playing it

```
gh pr checkout 57
# or manually:
git fetch origin pull/57/head:pr-57
git switch pr-57

# play it, then:
git switch main
git branch -D pr-57
```

Reading a diff tells you if the code is sane. Playing the branch tells you if the feature is good.
For games, do both.

18. Contributing a fix to an engine plugin you use

```
gh repo fork cool-dev/UnityInputHelper --clone
cd UnityInputHelper
git switch -c fix/gamepad-deadzone
# fix, commit
git push -u origin fix/gamepad-deadzone
gh pr create --repo cool-dev/UnityInputHelper
```

Maintainer asks for changes weeks later; sync first:

```
git fetch upstream
git rebase upstream/main
# make changes, commit
git push --force-with-lease
```

19. Tag and ship a build

```
git switch main && git pull
git tag -a v0.4.0 -m "Steam Next Fest demo build"
git push origin v0.4.0

gh release create v0.4.0 \
  --title "v0.4.0 - Next Fest Demo" \
  --notes "New: grapple hook, two levels. Fixed: save crash." \
  ./Builds/Game-v0.4.0-win64.zip
```

Six weeks later, a tester reports a bug in that exact build:

```
git switch --detach v0.4.0      # exactly that code
```

20. Onboarding a new artist to the repo

Send them this:

```
git clone --recurse-submodules git@github.com:studio/Game.git
cd Game
git lfs install
git lfs pull
```

Then the two rules that matter for a non-programmer:

```
# Before touching a scene or map:
git pull
git lfs lock Assets/Scenes/Level_02.unity

# After finishing:
git add Assets/Scenes/Level_02.unity
git commit -m "art: dress cabin interior, pass 2"
git push
git lfs unlock Assets/Scenes/Level_02.unity
```

Most artists are better served by a GUI (GitHub Desktop, Fork, Sourcetree, or the engine's built-in VC panel) than by the CLI. That's a completely legitimate choice — the commands underneath are identical.

21. "Your local changes would be overwritten by merge"

```
git stash -u
git pull
git stash pop
# resolve any conflicts now
```

Or commit first if the work is real, rather than stashing it.

22. You're in a weird state and want out

The universal escape hatches:

```
git merge --abort
git rebase --abort
git cherry-pick --abort
git revert --abort
git status          # ← always tells you which operation you're in and how to
```

And the nuclear option, when your local branch is a disaster and the server copy is fine:

```
git fetch origin
git reset --hard origin/main  # ⚠ destroys ALL local changes on this branch
```

Part 10 — Error message decoder

Message	Meaning	Fix
<code>fatal: not a git repository</code>	You're not inside a repo	<code>cd</code> to the right folder, or <code>git init</code>
Updates were rejected because the remote contains work you do not have	Someone pushed first	<code>git pull --rebase</code> then push
Your local changes would be overwritten by merge	Uncommitted edits collide with incoming ones	Commit or <code>git stash -u</code> first
<code>CONFLICT (content): Merge conflict in X</code>	Overlapping edits	Edit file, <code>git add</code> , continue
<code>CONFLICT (modify/delete)</code>	One side edited, other deleted	Decide: <code>git rm X</code> or <code>git add X</code>
You are in 'detached HEAD' state	On a commit, not a branch	<code>git switch -c name</code> to keep work, or <code>git switch main</code>
<code>fatal: refusing to merge unrelated histories</code>	Two repos with no common ancestor	<code>git merge --allow-unrelated-histories</code> (verify it's what you want)
<code>error: cannot lock ref</code>	Stale lock or branch/folder name clash	<code>git gc --prune=now</code> , or check for <code>feat</code> and <code>feat/x</code> both existing
Permission denied (publickey)	SSH key not set up	<code>ssh -T git@github.com</code> , add key to GitHub
<code>remote: Repository not found</code>	Wrong URL or no access	Check <code>git remote -v</code> and your permissions
This exceeds GitHub's file size limit of 100.00 MB	Big file without LFS	LFS-track it and rewrite the commit
Encountered N files that should have been pointers	LFS not installed when the file was	<code>git lfs install</code> , <code>git add --renormalize .</code>

	committed	
nothing to commit, working tree clean	Nothing changed — or it's all gitignored	git status --ignored to check
The following untracked working tree files would be overwritten	Incoming commit adds a file you already have locally	Move/delete your local copy, then pull

Part 11 — Cheat sheet

```
# --- Setup ---
git config --global user.name "Name"
git init
git clone <url>

# --- Daily ---
git status
git add <file>          git add -p
git commit -m "msg"     git commit -am "msg"
git log --oneline --graph --all
git diff                git diff --staged

# --- Branches ---
git switch -c <name>      git switch <name>
git branch -d <name>      git branch -a
git merge <name>         git rebase <name>

# --- Remotes ---
git fetch origin
git pull --rebase
git push -u origin <branch>
git push --force-with-lease
git push origin --delete <branch>

# --- Conflicts ---
git status
git checkout --ours <file>  git checkout --theirs <file>
git add <file>
git merge --continue       git merge --abort
git rebase --continue     git rebase --abort

# --- Undo ---
```

```

git restore <file>                # discard edits
git restore --staged <file>       # unstage
git commit --amend                # fix last commit
git reset --soft HEAD~1          # uncommit, keep staged
git reset --hard HEAD~1          # uncommit, destroy (⚠)
git revert <hash>                 # safe undo of pushed commit
git reflog                       # find anything you "lost"

# --- Rescue / investigate ---
git stash -u                      git stash pop
git cherry-pick <hash>
git bisect start / good / bad / reset
git log -S "searchTerm"
git blame <file>
git show <hash>:<file>

# --- Tags & builds ---
git tag -a v1.0 -m "msg"         git push origin v1.0
git switch --detach v1.0

# --- LFS ---
git lfs install
git lfs track "*.png"
git lfs lock <file>             git lfs locks      git lfs unlock <file>
git lfs migrate info

# --- Big repo ---
git worktree add ../hotfix main
git count-objects -vH
git clone --filter=blob:none <url>

```

Part 12 — Practice drills

Don't learn this on your real project. Make a sandbox:

```
mkdir git-practice && cd git-practice && git init
```

Drill 1 — the loop. Create three files, commit them separately. Use `git log --oneline` after each. Then `git add -p` a file with two separate edits and commit only one of them.

Drill 2 — cause a conflict on purpose. Commit a file with `speed = 10`. Branch, change it to `20`, commit. Switch back to main, change it to `5`, commit. Merge. Read the markers with

`zdiff3` on. Resolve. Do this three times until the markers stop feeling scary — this is the single highest-value drill in the list.

Drill 3 — the undo ladder. Make five commits. Then, restoring each time with `reflog : reset --soft HEAD~1 , reset --hard HEAD~2 , revert HEAD` . Notice which ones rewrite history and which add to it.

Drill 4 — clean up history. Make six sloppy commits (`wip , wip2 ...`). Squash them into two meaningful ones with `git rebase -i HEAD~6` .

Drill 5 — lose something and get it back. Commit work on a branch, switch away, delete the branch with `-D` . Recover it with `git reflog` . Feel the relief. Now you'll never truly panic again.

Drill 6 — the real thing. On your actual project: set up `.gitignore` and LFS correctly, push to GitHub, open a PR against your own repo, and merge it. Then tag a build and make a release.

Part 13 — Glossary

Term	Meaning
Repository (repo)	A project tracked by git — the files plus the <code>.git</code> folder of history
Commit	A snapshot of the project with a message, author, and parent
Hash / SHA	The unique ID of a commit, e.g. <code>a3f9c81</code>
HEAD	Pointer to your current position, usually a branch
Branch	A movable pointer to a commit
Working directory	The files as they currently exist on disk
Staging area / index	The set of changes queued for the next commit
Remote	A nickname for another copy of the repo (usually <code>origin</code>)
Remote-tracking branch	<code>origin/main</code> — your cached knowledge of the server's state
Upstream	The remote branch your local branch tracks; also, the original repo you forked from

Fast-forward	A merge that just slides the pointer forward — no merge commit needed
Three-way merge	A merge that compares both sides against their common ancestor
Merge commit	A commit with two parents, joining two lines of history
Conflict	Both sides changed the same region; git needs a human decision
Rebase	Replaying commits onto a new base, creating new commits with new hashes
Cherry-pick	Copying one commit's changes onto another branch
Stash	A temporary shelf for uncommitted work
Reflog	A local log of every position HEAD has held — the undo safety net
Tag	A permanent name for a specific commit, usually a release
Fork	Your own server-side copy of someone else's repo
Pull request (PR)	A GitHub request to merge one branch into another, with review
LFS	Large File Storage — keeps big binaries out of git history
Submodule	Another git repo pinned at a specific commit inside yours
Worktree	A second working directory sharing one repo's history
Bisect	Binary search through history to find the commit that broke something
Squash	Collapsing several commits into one
Detached HEAD	Being on a commit rather than a branch

The short version

1. Commit early and often — uncommitted work is the only work git can't save.
2. Branch for everything. Branches are free.
3. `git fetch` before you assume anything about the server.

4. `git status` when confused; it usually tells you the answer.
5. Never rebase or force-push shared history.
6. `git reflog` exists — you probably haven't lost it.
7. LFS and `.gitignore` from day one on a game project.
8. Binary assets get *locked*, not merged.
9. Tag every build you hand to another human.
10. Small PRs get real reviews. Big ones get rubber stamps.